STAGE 2 – PRACTICAL ENGINEERING CHALLENGE

TEAM ID:	TEAM NAME:
LRC-26-0550	Velvet Voltage

Problem Statement: Industrial Manufacturing Process Automations

Industrial manufacturing depends on highly controlled liquid-handling systems to deliver process fluids reliably from a source or bulk supply to downstream process equipment. Depending on the application, these fluids may include **ultrapure fluids, process chemicals, solvents, developers, or CMP-related fluids**. Industry standards explicitly address liquid-chemical distribution, high-purity piping, bulk chemical distribution, system qualification, and UPW system operation and monitoring.

In such systems, **flow is a fundamental process variable**. An incorrect flow rate can indicate or cause a restriction, incorrect valve state, inadequate pumping performance, abnormal process demand, or another hydraulic deviation. The consequence may be unstable process conditions, incorrect chemical delivery, equipment malfunction, or loss of process performance.

This challenge therefore focuses on a fundamental industrial facilities and process-engineering task: **To establish a defined process-fluid flow, measure it reliably, detect a deviation, diagnose its cause, and restore the process to the required operating condition.**

Engineering Challenge

As you are part of a multidisciplinary engineering team responsible for designing, building and validating a small-scale automated liquid-flow system represented of industrial process handling system.

CASE 1 – CONTROL + MEASUREMNT

Problem: Develop a system that manages the changing level of fluid in the sump automatically (when there is a sudden external fluid charging to the sump) and while accurately measuring and totalizing the fluid discharged from the sump using the flow transmitter.

CASE 2 – FLUSHING +SHUTDOWN

Problem: Develop an alternative operating configuration that allows flush water to pass through the system while all three pumps operate simultaneously, and then safely transition the system to an OFF condition when the flush-water supply has been exhausted.

Given Process Parameters

Pipe diameter =	**8mm**
Fluid density (ρ) =	**1000kg/m^3**
Dynamic Viscosity (μ) =	**1x10^{-3}Pa.s**
Required Flow range =	**1-6 L/min**
Nominal target flow for the live challenge =	**3L/min**

Available Hardware

Mechanical Hardware	Electronic Hardware
1. Water Tank – 3 Quantity 2. Pumps – 3 Quantity 3. FT-01 Flow Transmitter – 01 Quantity 4. Ball valve – 3 Quantity 5. ON/OFF/XV Valve – 1 Quantity 6. 10-mm OD tubing/blue – 9 meters/1 meter 7. Microcontroller/PCB/ data-acquisition software 8. Required electrical/ electronic accessories	1. Custom PCB 2. ESP32 NodeMCU module 3. Pump Driver-1 4. Pump Driver-2 5. Pump Driver-3 6. 12 V DC power input (**SMPS**) 7. 12 VDC Solenoid-valve interface 8. Flow-sensor interface 9. Float-switch interface 10. ON/OFF-switch interface 11. Pump-driver PWM/enable interfaces 12. On/Off Switch

The team must determine the appropriate arrangement, connections and measurement method.

- Assemble and commission the liquid-flow system using the components provided.
- Measure and characterize the flow using FT-01
- Interface FT-01 with the provided **data-acquisition system**
- Experimentally determine the relationship between sensor output and actual flow.
- Characterize the effect of valve position on flow rate. Record and analyse flow vs time data
- Maintain target flow of 3L/min
- Use graphs and measured data to justify engineering decisions
- Design an appropriate hydraulic arrangement and P&ID

TEAM INSTRUCTIONS

- As a team go through the list provided to read about each component that has been Listed.
- The PCB is a 12 V DC and the SMPS connections should be made carefully.

TASK 1 HYDRAULICS SYSTEM DESIGN

Q1.1 Design and build the hydraulic path with the flow transmitter on the recirculation line. Submit/sketch a labelled P&ID below by showing the pumps, Needle valves, flow transmitter, flow direction, Tank, sump, tube diameter and measurement points.

CASE 1 – Process Fluid Transfer & Automatic Level Protection

Objective: Set-up and operate a closed-loop process- fluid handling system using the available tank, pump, valves flow transmitter, sump, float sensor and control PCB. The System must continuously transfer process fluid recirculation system by connecting the 10L fluid tank with a On/Off valve to a pump-01 and the pump-01 goes to the inlet of sump 4.2L tank. The Sump 4.2L outlet line is connected to another pump-02 followed by a flow transmitter that measures the total flow back to the 10L fluid tank. The Pump-01 runs on 50% duty cycle and Pump-02 runs on 100% duty cycle. **Calibrate the flow transmitter** and demonstrate correct fluid circulation, if there is sudden introduction of external fluid into the Sump tank then Pump-01 should stop while pump – 02 should still runs till fluid level reduces in the sump. After the fluid reduction in the SUMP 4.2L tank the Pump-01 should turn on and the process must continue. **Also, demonstrate Flow vs Time trends in mL/s, total transferred Liters back to the 10L tank** (share Screenshots flow in L/min). **Share the code**. The case-1 can be turned on using the available on/off Switch, to start the system.

CASE 2 – Flush/Line Conditioning Sequence

Objective: After successfully completing case 1, initiate the flush water in the sequence using the **equal Tee.** The On/Off valve must be switched off in order to isolate the 10L process fluid tank. The flush Fluid tank (4.2L) is used as a temporary fluid source to flush out the process fluid or any debris from the system. Before the start of this process make sure you open the manual ball valve on the fluid flush line before Pump-03 starts. The entire process of case -1 and case - 2 should run in a single go without manual intervention except opening of ball valve before the Pump-03 starts. **The System must stop as soon as the tank flush fluid 4.2L tank finishes.** The system must be automated without human interventions (only ball valve can be opened manually before the start of flush line sequence)

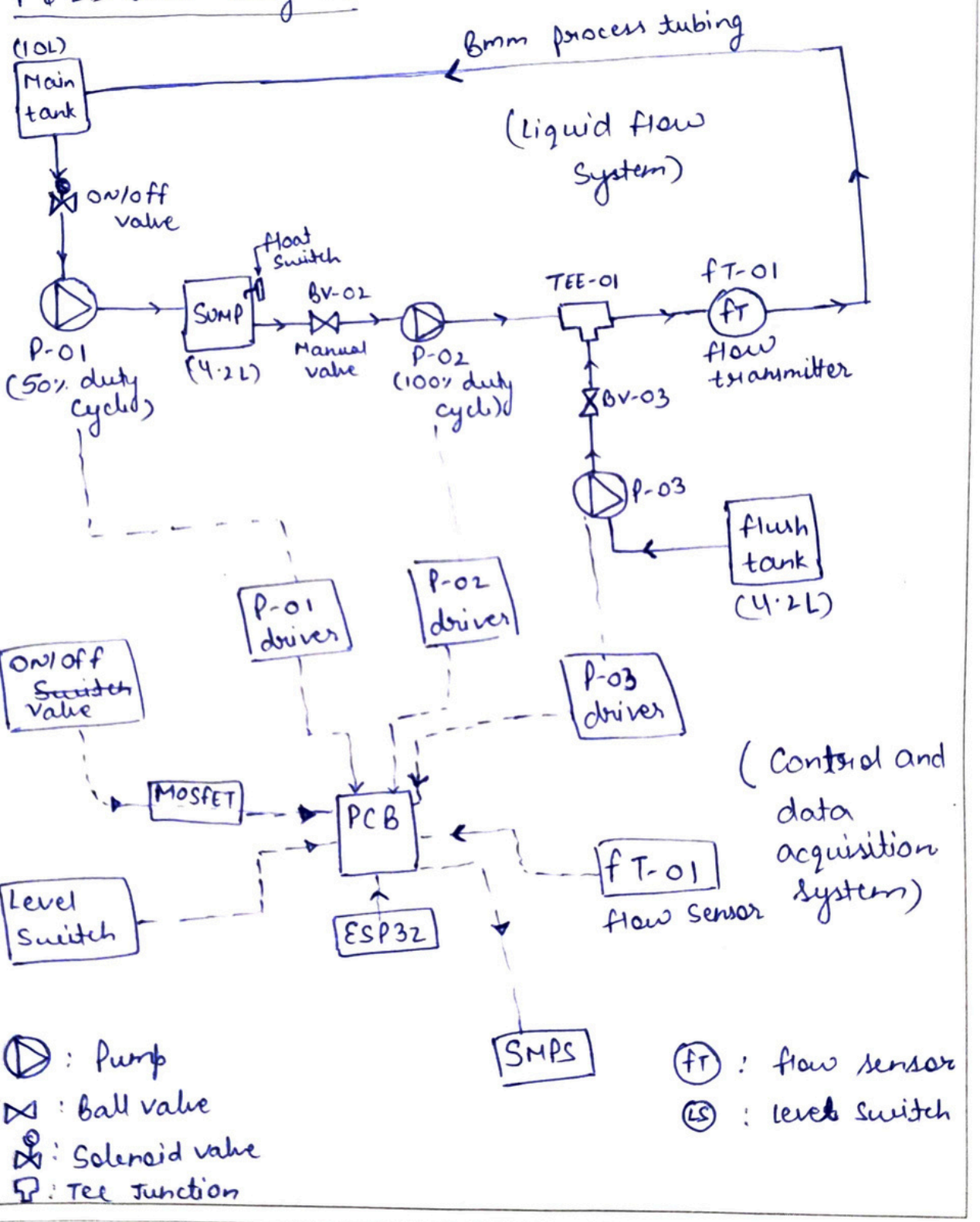
SUBMIT:

A. Submit: 10-15 min Video and Photograph of the completed setup, with identification of each component and explain the direction of flow in the video verbally. Demonstrate both Case - 1 and Case – 2.

B. Final P&ID Block diagram

P&ID SKETCH / WORKING SPACE

P & ID block diagram



Q1.2 After setting-up the system, now operate it at approximately at 1L/min, 3L/min & 6L/min. Compare the theoretical velocity with the actual operating behaviour.

WORKING / COMPARISON

$D = 8mm = 0.008\,m$

$\rho = 1000\ kg/m^3$

$Q = Av$

Q = discharge

A = Area of cross-section

v = velocity (theoretical)

$A = \frac{\pi D^2}{4} \Rightarrow A = 5.027 \times 10^{-5}\,m^2$

$Q(m^3/s) = \frac{Q(L/min) \times 10^{-3}}{60}$

1.) $Q = 1L/min = 1.667 \times 10^{-5}\,m^3/s$

$v = Q/A$, $\boxed{v = 0.332\ m/s}$

Expected actual behaviour:
Low flow and relatively low pressure drop.
In actual we found $v < 0.332 m/s$

2.) $Q = 3L/min \approx 5 \times 10^{-5}\,m^3/s$

$v = Q/A$, $v = 0.995 m/s$

Expected: Moderate flow, nominal target operating cond^n.

3.) $Q = 6L/min = 10 \times 10^{-5}\,m^3/s$

$v = Q/A$, $v = 1.989 m/s$

Expected: High flow with increased pressure drop and possible pump limitation

Actual behaviour

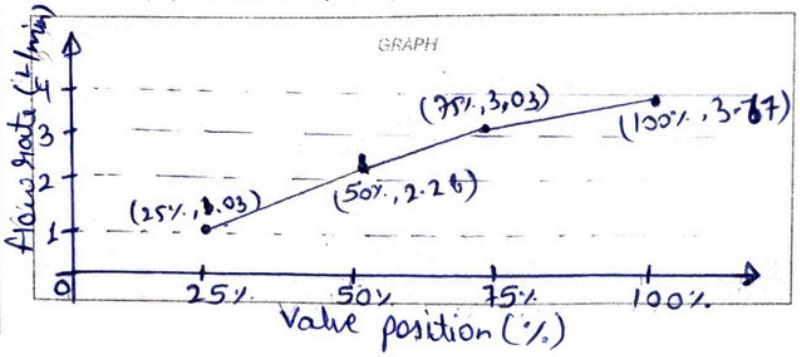
At the nominal target of 3L/min, the estimated average flow velocity is approx. 1.326 m/s assuming an 8mm dia internal pipe diameter

- At higher flow rates, pressure losses and pump limitations may become more significant
- Actual flow depends on valve posn, tubing resistance, and Hydraulic arrangements

TASK 2: FLOW RESPONSE TO VALVE POSITION

Q 2.1 Set the ball valve to approximately 25%,50%,75% & 100%. At each position, allow the flow to stabilize and record the flow rate. Create the following graph of valve position (%) vs Flow Rate (L/min)

VALVE POSITION	FLOW RATE
25%	1.03 L/min
50%	2.28 L/min
70%	3.03 L/min
100%	3.67 L/min



Q2.2 Determine the experimental calibration equation: $Q = af + b$, where Q = flow rate in L/min, f = pulse frequency in Hz, a = experimental calibration coefficient and b = offset.

Report:

R^2 =	0.9845
a =	0.01475
b =	− 0.15962
Maximum calibration error =	0.1715 L/min
Average calibration error =	0.1133 L/min

Q2.3 FT-01 produces pluses corresponding to water flow. Run the system at least for 5 different flow conditions within the available operating range (Hint: use formulas f=N/t and Q=V/t). For each condition, record the following:

Trial	Pulses	Time(s)	Frequency (Hz)	Collected Volume (L)	Actual Flow (L/min)
1	85	1 sec	85 Hz	0.019 L	1.14 L/min
2	170	1 sec	170 Hz	0.038 L	2.28 L/min
3	230	1 sec.	230 Hz	0.051 L	3.077 L/min
4	316	1 sec.	316 Hz	0.061 L	3.668 L/min
5	2270	10 sec.	226.8 Hz	0.513 L	3.037 L/min

Case-2

CALCULATIONS / OBSERVATIONS

Calculations

Q-2-2

Valve pos^n	f (Hz)	Q (L/min)
25%	76.9	1.03
50%	170.47	2.28
75%	226.47	3.03
100%	250	3.7
	$\Sigma f = 723.84$	$\Sigma Q = 10.04$

$\Sigma f^2 = 145786.55$

$\Sigma fQ = 2167.27$

$n = 4$

$Q = af + b$

for best-fit straight line, $Q = af + b$, use least-squares formulas

$$a = \frac{n\Sigma fQ - (\Sigma f)(\Sigma Q)}{n\Sigma f^2 - (\Sigma f)^2} \qquad b = \frac{\Sigma Q - a\Sigma f}{n}$$

Upon solving, $\boxed{a \approx 0.01475}$ $\boxed{b \approx -0.15962}$

final calibration equation, $\boxed{Q = 0.01475f - 0.15962}$

R^2 = coeff. of determination,

Calculated by, $R^2 = 1 - \frac{\Sigma(Q - Q')^2}{\Sigma(Q - \bar{Q})^2}$

Q = exp. flow rate

Q' = predicted flow rate by calibration eq^n

$\bar{Q}$ = avg. flow rate.

$\therefore \bar{Q} \approx 2.51$ L/min

Q	Q'	abs. error
1.03	0.97	0.055
2.28	2.35	0.075
3.03	3.18	0.15
3.7	3.52	0.172

$\therefore \Sigma(Q - Q')^2 = 0.06075$

$\Sigma(Q - \bar{Q})^2 = 3.9066$

$\therefore \boxed{R^2 \approx 0.9845}$

Q-2-3 $f = N/t$ $Q = V/t$

f = frequency
N = no. of pulses
t = time taken

Q = flow rate
V = volume
t = time taken

TASK 3: HYDRAULIC SYSTEM THEORY AND FAULT ANALYSIS

Q3.1 The effective liquid volume inside a section of tubing is 0.5L, while the operating flow rate is 2.5L/min. The hydraulic residence time is given by $T_r = Q/V$. Calculate the residence time in seconds, also answer the following questions:

1. What does residence time physically represent in this system?
2. What happens to residence time if the tubing volume is doubled while the flow rate remains constant?
3. What happens to residence time if the flow rate is doubled while the tubing volume remains constant?
4. Why is residence time important when interpreting a sudden change in measured flow?

ANSWER

$T_R = V/Q$

1.) $V = 0.5L$, $Q = 2.5\ L/min$

$T_R = \frac{0.5}{2.5} = 0.2\ min$

$\boxed{T_R = 12\ sec}$

2.) If $V = 2(0.5)L = 1L$, $Q = 2.5\ L/min$

$T_R = \frac{1}{2.5} = 0.4\ min$

$\boxed{T_R = 24\ sec.}$

Residence time doubles.

3.) If $V = 0.5L$, $Q = 2(2.5) = 5\ L/min$

$T_R = \frac{0.5}{5} = 0.1\ min$

$\boxed{T_R = 6\ sec.}$

Residence time becomes half.

4.) Residence time help us understand the delay between a change in hydraulic system and corresponding change is detected by fT-01. In our setup, water takes time to travel through the tubing, so a sudden change in pump operation or valve position may not appear immediately at fT-01. Understanding this delay helps distinguish normal response from possible faults such as pump failures, valve restrictions etc.

Q3.2 The following observation are obtained from the same system:

CONDITION	PUMP	VALVE	FT-01 PULSE SIGNAL	MEASURED FLOW
A	ON	Open	Normal	Normal
B	ON	Partially closed	Reduced	Reduced
C	ON	Open	Zero	Zero
D	OFF	Open	Zero	Zero
E	ON	Open	Intermittent	Fluctuating

For each of the condition, identify the most likely physical or instrumentation condition. Then answer: Which condition is most difficult to distinguish using FT-01 alone, and what additional information would you require to distinguish the fault? (Hint: Teams must consider possibilities such as: Valve restriction, pump failure, hydraulic blockage, sensor/signal failure, unstable flow etc)

ANSWER

Condition	Most likely Cause	Explanation
A	Normal operation	Pump is delivering fluid, FT-01 measuring flow correctly
B	Valve Restriction	Partially closing valve increases flow resistance, reduce flow rate and pulse frequency
C	Pump failure, Hydrolic blockage or FT-01 failure	Pump may not delivering fluid, the line may be blocked or the sensor may have failed.
D	Normal Shutdown	With the pump off, flow is expected to stop & FT-01 should produce zero pulses.
E	Unstable flow or intermittent sensor signal	Possible causes include air bubbles, unstable pump supply, loose wiring, or faulty sensor.

- condition C is most difficult to diagose. zero pulses and zero measured flow do not reveal whether the actual flow has stopped or whether FT-01 has failed.
- Additional information required:

① Pump status
② physical flow
③ Hydraulic blockage
④ sensor power supply
⑤ signal output
⑥ Independent Manual Measurements

HARDWARE LIST

Mechanical Hardware

Component	Description	Main Use
Ball Valve (BV-01)	A manually operated valve with a spherical shut-off mechanism.	Used to **manually isolate or permit fluid flow** in the pipeline. It is useful for maintenance, testing, and flow isolation.
10-Litre Fluid Tank (Fluid Tank-01)	Transparent storage tank with approximately 10 L capacity.	Used as the **main fluid reservoir** for storing process fluid. The transparent body also allows the fluid level and condition to be visually monitored.
5-Litre Sump	Small transparent reservoir labelled **SUMP**, approximately 4.5 L.	Used as a **collection/storage vessel** for fluid returning to or entering the sump section of the system.
4.2-Litre Flush Fluid Tank / Flush Line	The transparent vessel labelled **FLUSH FLUID** is approximately 4.5 L.	Provides **flush fluid** for cleaning or flushing the main pipe circuit when required.
ON/OFF Valve (OV-01)	Electrically operated valve/solenoid-type valve shown in the setup.	Used for **automatic control of fluid flow**, allowing the controller to turn a particular flow path on or off.
Pumps -Pump-1, Pump-2, Pump-3	Motor-driven fluid pumps. Three pumps are shown in the setup.	Used to **circulate and transfer fluid** between different sections of the system. Each pump can be independently controlled.
8 mm Process Pipe	Flexible blue tubing with approximately 8 mm process-line size.	Used as the **main fluid-flow path** connecting tanks, valves, pumps, tees, and other components.
10 mm × 8 mm Reducers	Reducer fittings that transition between a 10 mm connection and an 8 mm pipe.	Used to **adapt the pump connection size to the 8 mm process pipe. Mainly used for Pumps**
8 mm Connectors	Push-fit/barbed-type fittings compatible with the 8 mm tubing.	Used to **join pipes to components** and to make detachable connections between sections of tubing.
8 mm Equal Tee	T-shaped connector having three equal 8 mm ports.	Used to **split one 8 mm fluid line into two branches or combine two branches into one line.**

Electronic Hardware

Electronic Component	Identification / Function	Use
PCB -Printed Circuit Board	Custom control board	Provides the electrical interconnection between the **12 V power supply, ESP32 controller, sensors, valves, indicators, and pump-driver interfaces.**
ESP32 / NodeMCU ESP32S module (U7)	Main microcontroller	Acts as the **central controller.** It provides GPIO/PWM signals for controlling outputs and receiving signals from sensors/switches.

Electronic Component	Identification / Function	Use
12 V DC Input Connector	Marked **12V INPUT**	Connection point for the external 12 V DC power supply.
12 V Output Connectors	P1, P2, P3 and additional 12 V output terminals	Provide 12 V power connections for external loads such as the pump-driver/motor circuits.
Motor Driver Interfaces -U2, U3, U4	Three multi-pin driver connectors shown on the PCB	Provide dedicated interfaces for **Pump-1, Pump-2 and Pump-3 driver/control connections**.
Solenoid Valve Connector	Labelled **SOLENOID_VALVE -S/V/G**	Interface for controlling/connecting the solenoid or ON/OFF valve.
Flow Sensor Connector	Labelled **FLOW_SENSOR - S/V/G**	Interface for the water-flow sensor; supplies power and receives the sensor signal. **THE FLOW SENSOR SHOULD BE CALIBRATED**
Float Switch Connector	Labelled **FLOAT_SWITCH -FSW/GND**	Interface for a float/level switch used to detect a water-level condition.
ON/OFF Switch Connector	Labelled **ON/OFF_SWITCH -SW/GND**	Interface for an external manual ON/OFF switch.
12V DC SMPS	**12V DC SMPS**	For Power Supply to the board

Calibration Factor calculation code

```
// PIN DEFINITIONS

// Pump 1 - BTS7960
#define P1_L_PWM 32
#define P1_R_PWM 33
#define P1_R_EN  26
#define P1_L_EN  13

// Pump 2 - BTS7960
#define P2_L_PWM 23
#define P2_R_PWM 25
#define P2_R_EN  22
#define P2_L_EN  21

// FT-01 flow sensor
#define FLOW_PIN 14

//  PWM SETTINGS

const int PWM_FREQUENCY = 1000;
const int PWM_RESOLUTION = 8;

// Pump 1 = 50%, Pump 2 = 100%
const int PUMP1_PWM = 128;
const int PUMP2_PWM = 255;

// Set this to the actual measured water volume
const float KNOWN_VOLUME_L = 1.0f;

// FLOW VARIABLES

volatile uint32_t pulseCount = 0;

bool calibrationRunning = false;

unsigned long calibrationStartTime = 0;

// FLOW INTERRUPT

void IRAM_ATTR flowPulseISR() {
  pulseCount++;
}

//  PUMP 1

void startPump1() {
  digitalWrite(P1_R_PWM, LOW);

  digitalWrite(P1_R_EN, HIGH);
  digitalWrite(P1_L_EN, HIGH);

  ledcWrite(P1_L_PWM, PUMP1_PWM);
}

void stopPump1() {
  ledcWrite(P1_L_PWM, 0);

  digitalWrite(P1_R_EN, LOW);
  digitalWrite(P1_L_EN, LOW);
}
```

```
//PUMP 2

void startPump2() {
  digitalWrite(P2_R_PWM, LOW);

  digitalWrite(P2_R_EN, HIGH);
  digitalWrite(P2_L_EN, HIGH);

  ledcWrite(P2_L_PWM, PUMP2_PWM);
}

void stopPump2() {
  ledcWrite(P2_L_PWM, 0);

  digitalWrite(P2_R_EN, LOW);
  digitalWrite(P2_L_EN, LOW);
}

//  STOP BOTH PUMPS

void stopBothPumps() {
  stopPump1();
  stopPump2();
}

// SETUP

void setup() {
  Serial.begin(115200);

  // Motor driver pins
  pinMode(P1_R_PWM, OUTPUT);
  pinMode(P1_R_EN, OUTPUT);
  pinMode(P1_L_EN, OUTPUT);

  pinMode(P2_R_PWM, OUTPUT);
  pinMode(P2_R_EN, OUTPUT);
  pinMode(P2_L_EN, OUTPUT);

  // Flow sensor
  pinMode(FLOW_PIN, INPUT);

  // Keep drivers disabled during initialization
  stopBothPumps();

  // Initialize PWM (ESP32 Arduino Core 3.x)
  bool pwm1OK = ledcAttach(
    P1_L_PWM, PWM_FREQUENCY, PWM_RESOLUTION
  );

  bool pwm2OK = ledcAttach(
    P2_L_PWM, PWM_FREQUENCY, PWM_RESOLUTION
  );

  if (!pwm1OK || !pwm2OK) {
    Serial.println("PWM INITIALIZATION FAILED");
    stopBothPumps();

    while (true) {
      delay(1000);
    }
  }


}
```

```
  ledcWrite(P1_L_PWM, 0);
  ledcWrite(P2_L_PWM, 0);

  // Count falling edges from FT-01
  attachInterrupt(
    digitalPinToInterrupt(FLOW_PIN),
    flowPulseISR,
    FALLING
  );

  Serial.println();
  Serial.println("================================");
  Serial.println("FT-01 FLOW CALIBRATION");
  Serial.println("PUMP 1: 50% PWM");
  Serial.println("PUMP 2: 100% PWM");
  Serial.println("================================");

  Serial.print("Known volume: ");
  Serial.print(KNOWN_VOLUME_L, 3);
  Serial.println(" L");

  Serial.println("Send S to start both pumps.");
  Serial.println("Send E to stop and calculate factor.");
  Serial.println("================================");
}

// MAIN LOOP

void loop() {
  if (!Serial.available()) {
    return;
  }

  char command = Serial.read();

  // START CALIBRATION AND PUMPS
  if ((command == 'S' || command == 's')
      && !calibrationRunning) {

    // Reset pulse counter
    noInterrupts();
    pulseCount = 0;
    interrupts();

    calibrationStartTime = millis();

    // Start both pumps
    startPump1();
    startPump2();

    calibrationRunning = true;
    Serial.println();
    Serial.println("CALIBRATION STARTED");
    Serial.println("Pump 1: ON - 50% PWM");
    Serial.println("Pump 2: ON - 100% PWM");
    Serial.println("Collect the known volume of water.");
  }

  // STOP PUMPS AND CALCULATE CALIBRATION FACTOR
  else if ((command == 'E' || command == 'e')
          && calibrationRunning) {

    // Stop pumps first
    stopBothPumps();
```

```
    unsigned long elapsedTime =
      millis() - calibrationStartTime;

    // Read pulse count safely
    noInterrupts();
    uint32_t measuredPulses = pulseCount;
    interrupts();

    calibrationRunning = false;

    Serial.println();
    Serial.println("========== RESULTS ==========");

    Serial.println("Pump 1: OFF");
    Serial.println("Pump 2: OFF");

    Serial.print("Measured volume: ");
    Serial.print(KNOWN_VOLUME_L, 3);
    Serial.println(" L");

    Serial.print("Total pulses: ");
    Serial.println(measuredPulses);

    Serial.print("Elapsed time: ");
    Serial.print(elapsedTime / 1000.0f, 2);
    Serial.println(" seconds");

    if (KNOWN_VOLUME_L > 0.0f && measuredPulses > 0) {
      float calibrationFactor =
        measuredPulses / KNOWN_VOLUME_L;

      Serial.print("CALIBRATION FACTOR: ");
      Serial.print(calibrationFactor, 2);
      Serial.println(" pulses/L");

      Serial.println();
      Serial.println("Use this value in your main code:");
      Serial.print("PULSES_PER_LITER = ");
      Serial.print(calibrationFactor, 2);
      Serial.println(";");
    } else {
      Serial.println(
        "ERROR: Invalid volume or no pulses detected."
      );
      Serial.println(
        "Check the sensor wiring and signal."
      );
    }

    Serial.println("==============================");
    Serial.println("Send S to calibrate again.");
  }
}
```

CASE 1 CODE

```
//  PIN DEFINITIONS

// PUMP 1 - BTS7960
#define P1_L_PWM 32
#define P1_R_PWM 33
#define P1_R_EN  26
#define P1_L_EN  13

// PUMP 2 - BTS7960
#define P2_L_PWM 23
#define P2_R_PWM 25
#define P2_R_EN  22
#define P2_L_EN  21

// SOLENOID
#define VALVE_PIN 27

// FLOAT SENSOR
#define FLOAT_PIN 35

// TACTILE SWITCH
#define SWITCH_PIN 34

// FT-01 FLOW TRANSMITTER
#define FLOW_PIN 14

// PWM SETTINGS

const int PUMP1_PWM = 128;
const int PUMP2_PWM = 255;

const int PWM_FREQUENCY = 1000;
const int PWM_RESOLUTION = 8;

//  PUMP 2 TIMING

const unsigned long P2_ON_TIME  = 5200;
const unsigned long P2_OFF_TIME = 4800;

//  FLOAT SENSOR

const int HIGH_LEVEL_STATE = LOW;

//  FLOW CALIBRATION

const float FLOW_PULSES_PER_LITER = 4480.0f;

// Sampling interval

const unsigned long FLOW_INTERVAL = 1000;

//VARIABLES

bool systemRunning = false;
bool highLevelDetected = false;
bool pump2Running = false;

unsigned long pump2Timer = 0;
unsigned long serialTimer = 0;

//  FLOW VARIABLES

volatile uint32_t flowPulseCount = 0;

portMUX_TYPE flowMux = portMUX_INITIALIZER_UNLOCKED;

uint32_t lastFlowPulseCount = 0;

unsigned long lastFlowTime = 0;
```

```
float flowRateLMin = 0.0f;
float flowFrequency = 0.0f;

uint64_t totalRunPulses = 0;

float totalWaterLitres = 0.0f;

//  SWITCH DEBOUNCE

int lastSwitchReading = HIGH;
int stableSwitchState = HIGH;

unsigned long debounceTimer = 0;

const unsigned long DEBOUNCE_TIME = 50;

//  FLOW INTERRUPT

void IRAM_ATTR flowPulseISR()
{
 portENTER_CRITICAL_ISR(&flowMux);
 flowPulseCount++;
 portEXIT_CRITICAL_ISR(&flowMux);
}

//  READ FLOW PULSES

uint32_t readFlowPulses()
{
 uint32_t count;

 portENTER_CRITICAL(&flowMux);
 count = flowPulseCount;
 portEXIT_CRITICAL(&flowMux);

 return count;
}

//  UPDATE FLOW

void updateFlow()
{
 unsigned long currentTime = millis();

 unsigned long elapsed = currentTime - lastFlowTime;

 if (elapsed < FLOW_INTERVAL)
  return;

 uint32_t currentPulses = readFlowPulses();

 uint32_t pulseDifference =
  currentPulses - lastFlowPulseCount;

 lastFlowPulseCount = currentPulses;
 lastFlowTime = currentTime;

 if (!systemRunning)
 {
  flowRateLMin = 0;
  flowFrequency = 0;
  return;
 }

 // Frequency in Hz
 flowFrequency =
  (pulseDifference * 1000.0f) / elapsed;

 // Current flow rate in litres/minute
 flowRateLMin =
  (flowFrequency * 60.0f) /
  FLOW_PULSES_PER_LITER;
```

```
 // Total water since START
  totalRunPulses += pulseDifference;

  totalWaterLitres =
    (double)totalRunPulses /
    FLOW_PULSES_PER_LITER;
}

//  PUMP 1
void startPump1()
{
  digitalWrite(P1_R_PWM, LOW);

  digitalWrite(P1_R_EN, HIGH);
  digitalWrite(P1_L_EN, HIGH);

  ledcWrite(P1_L_PWM, PUMP1_PWM);
}

void stopPump1()
{
  ledcWrite(P1_L_PWM, 0);

  digitalWrite(P1_R_EN, LOW);
  digitalWrite(P1_L_EN, LOW);
}

// PUMP 2

void startPump2()
{
  digitalWrite(P2_R_PWM, LOW);

  digitalWrite(P2_R_EN, HIGH);
  digitalWrite(P2_L_EN, HIGH);

  ledcWrite(P2_L_PWM, PUMP2_PWM);
}

void stopPump2()
{
  ledcWrite(P2_L_PWM, 0);

  digitalWrite(P2_R_EN, LOW);
  digitalWrite(P2_L_EN, LOW);
}

//  START SYSTEM

void startSystem()
{
  if (systemRunning)
    return;

  // Reset flow readings for new run
  lastFlowPulseCount = readFlowPulses();

  lastFlowTime = millis();

  flowRateLMin = 0;
  flowFrequency = 0;

  totalRunPulses = 0;
  totalWaterLitres = 0;

  systemRunning = true;

  digitalWrite(VALVE_PIN, HIGH);

  highLevelDetected =
    (digitalRead(FLOAT_PIN) == HIGH_LEVEL_STATE);
```

```
 if (highLevelDetected)
 {
   stopPump1();
   startPump2();
 }
 else
 {
   startPump1();
   startPump2();
 }

 pump2Running = true;
 pump2Timer = millis();

 Serial.println("SYSTEM STARTED");
 Serial.println("FLOW TOTAL RESET");
}

//STOP SYSTEM

void stopSystem()
{
 if (!systemRunning)
   return;

 stopPump1();
 stopPump2();

 digitalWrite(VALVE_PIN, LOW);

 // Capture remaining pulses since last sample
 uint32_t currentPulses = readFlowPulses();

 totalRunPulses +=
   (uint32_t)(currentPulses - lastFlowPulseCount);

 lastFlowPulseCount = currentPulses;

 totalWaterLitres =
   (double)totalRunPulses /
   FLOW_PULSES_PER_LITER;

 systemRunning = false;
 pump2Running = false;

 flowRateLMin = 0;
 flowFrequency = 0;

 lastFlowTime = millis();
 Serial.println("SYSTEM STOPPED");

  Serial.print("FINAL WATER VOLUME: ");
  Serial.print(totalWaterLitres, 3);
  Serial.println(" L");
}

// NORMAL LEVEL

void normalLevelControl()
{
 startPump1();

 unsigned long currentTime = millis();

 if (pump2Running)
 {
   if (currentTime - pump2Timer >= P2_ON_TIME)
   {
     stopPump2();

     pump2Running = false;
     pump2Timer = currentTime;

     Serial.println("PUMP 2 OFF");
   }
 }
```

```
else
 {
  if (currentTime - pump2Timer >= P2_OFF_TIME)
  {
   startPump2();

   pump2Running = true;
   pump2Timer = currentTime;

   Serial.println("PUMP 2 ON");
  }
 }
}

// HIGH WATER LEVEL

void highLevelControl()
{
 stopPump1();

 startPump2();

 pump2Running = true;
 pump2Timer = millis();
}

//  SETUP

void setup()
{
 Serial.begin(115200);

 pinMode(P1_R_PWM, OUTPUT);
 pinMode(P1_R_EN, OUTPUT);
 pinMode(P1_L_EN, OUTPUT);

 pinMode(P2_R_PWM, OUTPUT);
 pinMode(P2_R_EN, OUTPUT);
 pinMode(P2_L_EN, OUTPUT);

 pinMode(VALVE_PIN, OUTPUT);

 pinMode(FLOAT_PIN, INPUT);
 pinMode(SWITCH_PIN, INPUT);

 // FT-01 pulse input.
 // External signal conditioning/pull-up
 // may be required depending on transmitter.
 pinMode(FLOW_PIN, INPUT);

 // Drivers OFF initially
 digitalWrite(P1_R_EN, LOW);
 digitalWrite(P1_L_EN, LOW);

 digitalWrite(P2_R_EN, LOW);
 digitalWrite(P2_L_EN, LOW);

 digitalWrite(P1_R_PWM, LOW);
 digitalWrite(P2_R_PWM, LOW);

 digitalWrite(VALVE_PIN, LOW);

 bool pwm1OK = ledcAttach(
  P1_L_PWM,
  PWM_FREQUENCY,
  PWM_RESOLUTION
 );
```

```
  bool pwm2OK = ledcAttach(
    P2_L_PWM,
    PWM_FREQUENCY,
    PWM_RESOLUTION
  );

  if (!pwm1OK || !pwm2OK)
  {
    Serial.println("PWM INITIALIZATION FAILED");

    while (true)
    {
      delay(1000);
    }
  }

  ledcWrite(P1_L_PWM, 0);
  ledcWrite(P2_L_PWM, 0);

  // Count falling edges from FT-01.
  // Change to RISING if required by its
  // verified output signal characteristics.
  attachInterrupt(
    digitalPinToInterrupt(FLOW_PIN),
    flowPulseISR,
    FALLING
  );

  lastFlowPulseCount = readFlowPulses();
  lastFlowTime = millis();

  lastSwitchReading = digitalRead(SWITCH_PIN);
  stableSwitchState = lastSwitchReading;


  Serial.println();
  Serial.println("================================");
  Serial.println("LAM RESEARCH - CASE 1");
  Serial.println("ESP32 CONTROL READY");
  Serial.println("================================");

  Serial.println("Pump 1: 50%");
  Serial.println("Pump 2: 100%");
  Serial.println("Pump 2: 5.2s ON / 4.8s OFF");

  Serial.println("Float: GPIO35");
  Serial.println("Switch: GPIO34");
  Serial.println("Solenoid: GPIO27");
  Serial.println("FT-01: GPIO14");

  Serial.println("Flow measurement enabled");
  Serial.println("Calibrate pulses per litre");

  Serial.println("Press switch to START");
}

//  MAIN LOOP

void loop()
{
  unsigned long currentTime = millis();

  //  TACTILE SWITCH

  int reading = digitalRead(SWITCH_PIN);

  if (reading != lastSwitchReading)
  {
    debounceTimer = currentTime;
  }
```

```
  if (currentTime - debounceTimer >= DEBOUNCE_TIME)
  {
    if (reading != stableSwitchState)
    {
      stableSwitchState = reading;

      if (stableSwitchState == LOW)
      {
        if (systemRunning)
          stopSystem();
        else
          startSystem();
      }
    }
  }

  lastSwitchReading = reading;

  //  FLOAT SENSOR

  highLevelDetected =
    (digitalRead(FLOAT_PIN) == HIGH_LEVEL_STATE);

  //  CONTROL

  if (systemRunning)
  {
    if (highLevelDetected)
      highLevelControl();
    else
      normalLevelControl();
  }

  //  FLOW MEASUREMENT

  updateFlow();

  //  SERIAL MONITOR

  if (currentTime - serialTimer >= 1000)
  {
    serialTimer = currentTime;


    Serial.println();
    Serial.println("================================");

    Serial.print("System: ");
    Serial.println(
      systemRunning ? "RUNNING" : "STOPPED"
    );

    Serial.print("Float: ");
    Serial.println(
      highLevelDetected ? "HIGH LEVEL" : "NORMAL"
    );

    Serial.print("Pump 1: ");

    if (!systemRunning || highLevelDetected)
      Serial.println("OFF");
    else
      Serial.println("ON - 50% PWM");

    Serial.print("Pump 2: ");


}
```

```
  if (!systemRunning)
    Serial.println("OFF");
  else if (highLevelDetected)
    Serial.println("ON - 100% CONTINUOUS");
  else if (pump2Running)
    Serial.println("ON - 100%");
  else
    Serial.println("OFF - TIMER");

  Serial.print("Solenoid: ");
  Serial.println(
    systemRunning ? "ON" : "OFF"
  );

  Serial.println("---------- FT-01 ----------");

  Serial.print("Frequency: ");
  Serial.print(flowFrequency, 1);
  Serial.println(" Hz");

  Serial.print("Flow Rate: ");
  Serial.print(flowRateLMin, 3);
  Serial.println(" L/min");

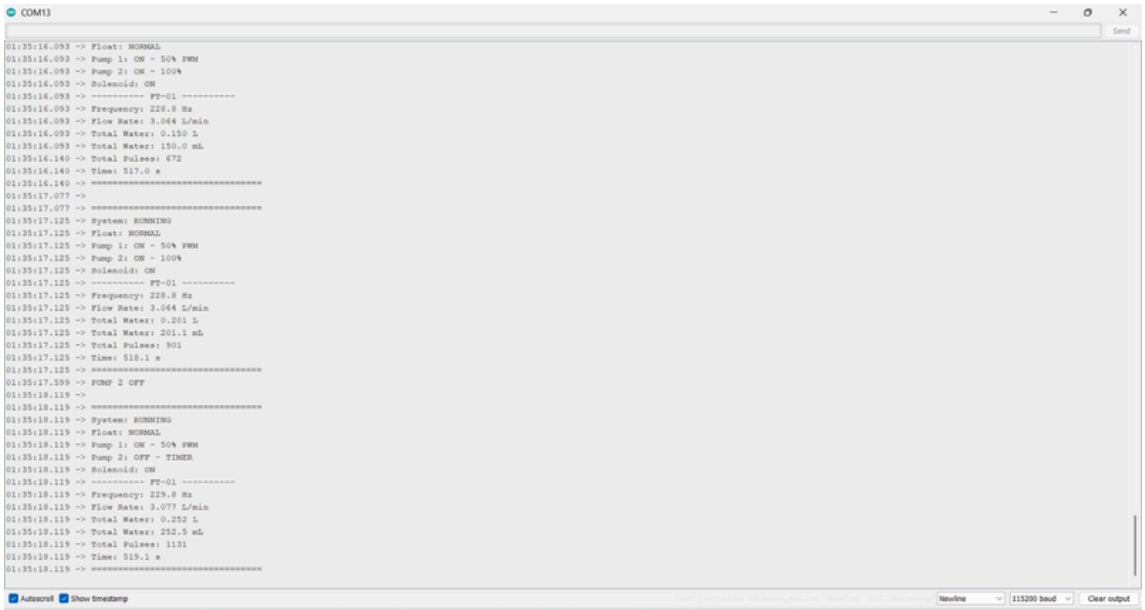
  Serial.print("Total Water: ");
  Serial.print(totalWaterLitres, 3);
  Serial.println(" L");

  Serial.print("Total Water: ");
  Serial.print(totalWaterLitres * 1000.0f, 1);
  Serial.println(" mL");

  Serial.print("Total Pulses: ");
  Serial.println((unsigned long)totalRunPulses);

  Serial.print("Time: ");
  Serial.print(currentTime / 1000.0, 1);
  Serial.println(" s");

  Serial.println("================================");
 }
}
```



SERIAL MONITOR READINGS

CASE 2 CODE

```
// Motor Driver 1

#define P1_L_PWM 32
#define P1_R_PWM 33
#define P1_R_EN  26
#define P1_L_EN  13


// Motor Driver 3

#define P2_L_PWM 23
#define P2_R_PWM 25
#define P2_R_EN  22
#define P2_L_EN  21


// Motor Driver 2
// R_PWM2 held LOW externally

#define P3_L_PWM 18
#define P3_R_EN  17
#define P3_L_EN  16



#define SOLENOID_PIN 27
#define FLOW_PIN     14
#define SWITCH_PIN   34


#define PWM_FREQ 1000
#define PWM_RES  8

const int P1_SPEED = 128;
const int P2_SPEED = 255;
const int P3_SPEED = 255;

// FLUSH SETTINGS

const float FLUSH_VOLUME_L = 4.0;
const float EXPECTED_FLOW_LMIN = 2.6;

// Provisional FT-01 calibration coefficient.
// Replace with your measured calibration value.
const float PULSES_PER_LITER = 5169.23;

// 85-second shutdown
const unsigned long MAX_FLUSH_TIME_MS = 85000;

// VARIABLES

volatile unsigned long pulseCount = 0;

bool flushing = false;
bool finished = false;

unsigned long flushStartTime = 0;
unsigned long lastFlowTime = 0;

float flowRateLMin = 0.0;
float totalLiters = 0.0;
float frequencyHz = 0.0;

// SWITCH DEBOUNCE

int lastSwitchReading = HIGH;
int stableSwitchState = HIGH;

unsigned long debounceTime = 0;

const unsigned long DEBOUNCE_MS = 50;
```

```
// FT-01 INTERRUPT


void IRAM_ATTR flowPulse()
{
  pulseCount++;
}

// PUMP 1

void startPump1()
{
  digitalWrite(P1_R_PWM, LOW);

  digitalWrite(P1_R_EN, HIGH);
  digitalWrite(P1_L_EN, HIGH);

  ledcWrite(P1_L_PWM, P1_SPEED);
}

void stopPump1()
{
  ledcWrite(P1_L_PWM, 0);

  digitalWrite(P1_R_EN, LOW);
  digitalWrite(P1_L_EN, LOW);
}

// PUMP 2

void startPump2()
{
  digitalWrite(P2_R_PWM, LOW);

  digitalWrite(P2_R_EN, HIGH);
  digitalWrite(P2_L_EN, HIGH);

  ledcWrite(P2_L_PWM, P2_SPEED);
}

void stopPump2()
{
  ledcWrite(P2_L_PWM, 0);

  digitalWrite(P2_R_EN, LOW);
  digitalWrite(P2_L_EN, LOW);
}

// PUMP 3
void startPump3()
{
  digitalWrite(P3_R_EN, HIGH);
  digitalWrite(P3_L_EN, HIGH);

  ledcWrite(P3_L_PWM, P3_SPEED);
}

void stopPump3()
{
  ledcWrite(P3_L_PWM, 0);

  digitalWrite(P3_R_EN, LOW);
  digitalWrite(P3_L_EN, LOW);
}
```

```
// STOP SYSTEM

void stopSystem(const char* reason)
{
  stopPump1();
  stopPump2();
  stopPump3();

  // Keep process tank isolated
  digitalWrite(SOLENOID_PIN, LOW);

  flushing = false;
  finished = true;

  Serial.println();
  Serial.println("================================");
  Serial.println("CASE 2 - SYSTEM SHUTDOWN");

  Serial.print("Reason: ");
  Serial.println(reason);

  Serial.println("Pump 1: OFF");
  Serial.println("Pump 2: OFF");
  Serial.println("Pump 3: OFF");

  Serial.print("Total measured volume: ");
  Serial.print(totalLiters, 3);
  Serial.println(" L");

  Serial.println("================================");
}


// START FLUSHING


void startFlushing()
{
  if (flushing || finished)
  {
    return;
  }

  // Close process-tank valve
  digitalWrite(SOLENOID_PIN, LOW);

  // Reset flow pulse count
  noInterrupts();
  pulseCount = 0;
  interrupts();

  flowRateLMin = 0.0;
  totalLiters = 0.0;
  frequencyHz = 0.0;

  flushStartTime = millis();
  lastFlowTime = flushStartTime;

  // Start all three pumps
  startPump1();
  startPump2();
  startPump3();

  flushing = true;
```

```
  Serial.println();
  Serial.println("================================");
  Serial.println("CASE 2 - FLUSHING STARTED");
  Serial.println("Pump 1: 50%");
  Serial.println("Pump 2: 100%");
  Serial.println("Pump 3: 100%");
  Serial.println("Solenoid: LOW");
  Serial.println("Shutdown timer: 85 seconds");
  Serial.println("================================");
}
```



SERIAL MONITOR READINGS